

Architecture Design Report

This document presents the architectural design of the system, including its context, logical structure, runtime behavior, and deployment.

1. System Requirements

System Name: LibraryManagementSystem

Architecture Style: Client Server

Functional Requirements

- The system must analyze gestures and reflect the corresponding action on the projected surface.
- The system will include gesture recognition.
- The system will include face recognition.
- The system will include hand tracking.
- The system will include single-hand interaction.
- The system will include customizing the actions for specific gestures.
- The system must enable real-time collaboration by supporting interactions on the projected surface.
- The system must have low-latency.
- The system must work effectively in different places.
- The system must be able to detect the user's face using the YOLOv5 model.
- The system must be able to detect the borders of objects using contours detection.
- The system must be able to mute the image noise using Gaussian blur.
- The system must be able to separate the moving parts from the background using Background Subtraction.
- The system must be able to optimize the image contrast using Histogram Equalization.
- The system must enable concurrent execution of tasks within a single process using threading.
- The system must enable parallel execution of tasks across multiple processes using multiprocessing.

Non-Functional Requirements

- The system must have a competitive price.

Description: The system must have a competitive price. After making a competitive analysis we figured out that the system must be able to handle multiple users.

- The system must be able to handle multiple users.

Description: The system must be able to handle multiple users. MUST handle multiple users.

- The system must be able to handle large amounts of data.

Description: The system must be able to handle large amounts of data. MUST handle large amounts of data.

- The system must be able to provide a user-friendly interface.

Description: The system must be able to provide a user-friendly interface. SHOULD provide a user

- The system must be able to provide real-time feedback to the user.

Description: The system must be able to provide real-time feedback to the user. SHOULD provide real-time

- The system must be able to handle errors and exceptions.

Description: The system must be able to handle errors and exceptions. MUST handle errors and exceptions

- The system must be able to provide security features.

Description: The system must be able to provide security features. SHOULD provide security features.

- The system must be able to provide scalability.

Description: The system must be able to provide scalability. SHOULD provide scalability.

2. Formal Architecture Specification (ACME ADL)

The following section provides a formal architectural specification using ACME ADL, capturing components, con

```
System LibraryManagementSystem {
  Property view : String = "runtime";
  Property architectureStyle : String = "Client server";
  Property source : String = "AI";
  Component GestureRecognitionModule = new Component {
    Port in;
    Port out;
    Property responsibility : String = "Analyze gestures and reflect the corresponding action on the";
    Property state : String = "stateless";
    Property scaling : String = "horizontal";
  }
  Component FaceRecognitionModule = new Component {
    Port in;
    Port out;
    Property responsibility : String = "Detect the user's face using the YOLOv5 model.";
    Property state : String = "stateless";
    Property scaling : String = "horizontal";
  }
  Component HandTrackingModule = new Component {
    Port in;
    Port out;
    Property responsibility : String = "Track the user's hand movements.";
    Property state : String = "stateless";
    Property scaling : String = "horizontal";
  }
  Component Single-HandInteractionModule = new Component {
    Port in;
    Port out;
    Property responsibility : String = "Enable single-hand interaction with the projected surface.";
    Property state : String = "stateless";
    Property scaling : String = "horizontal";
  }
}
```

```

Component CustomizationModule = new Component {
    Port in;
    Port out;
    Property responsibility : String = "Customize the actions for specific gestures.";
    Property state : String = "stateless";
    Property scaling : String = "horizontal";
}
Component Real-TimeCollaborationModule = new Component {
    Port in;
    Port out;
    Property responsibility : String = "Enable real-time collaboration by supporting interactions on";
    Property state : String = "stateless";
    Property scaling : String = "horizontal";
}
Component ImageProcessingModule = new Component {
    Port in;
    Port out;
    Property responsibility : String = "['Detect the borders of objects using contours detection.',
    Property state : String = "stateless";
    Property scaling : String = "horizontal";
}
Component BackgroundSubtractionModule = new Component {
    Port in;
    Port out;
    Property responsibility : String = "Separate the moving parts from the background using Background";
    Property state : String = "stateless";
    Property scaling : String = "horizontal";
}
Component ThreadingModule = new Component {
    Port in;
    Port out;
    Property responsibility : String = "Enable concurrent execution of tasks within a single process";
    Property state : String = "stateless";
    Property scaling : String = "horizontal";
}
Component MultiprocessingModule = new Component {
    Port in;
    Port out;
    Property responsibility : String = "Enable parallel execution of tasks across multiple processes";
    Property state : String = "stateless";
    Property scaling : String = "horizontal";
}
Component LatencyOptimizationModule = new Component {
    Port in;
    Port out;
    Property responsibility : String = "Ensure low-latency performance.";
    Property state : String = "stateless";
    Property scaling : String = "horizontal";
}
Component EnvironmentalAdaptationModule = new Component {
    Port in;
    Port out;
    Property responsibility : String = "Work effectively in different places.";
    Property state : String = "stateless";
    Property scaling : String = "horizontal";
}
Attachment GestureRecognitionModule.out to HandTrackingModule.in {
    Property delivery : String = "at-least-once";
}

```

```

    Property ordering : Boolean = false;
    Property backpressure : String = "bounded";
}
Attachment HandTrackingModule.out to Single-HandInteractionModule.in {
    Property delivery : String = "at-least-once";
    Property ordering : Boolean = true;
    Property backpressure : String = "bounded";
}
Attachment FaceRecognitionModule.out to HandTrackingModule.in {
    Property delivery : String = "at-least-once";
    Property ordering : Boolean = false;
    Property backpressure : String = "bounded";
}
Attachment HandTrackingModule.out to ImageProcessingModule.in {
    Property delivery : String = "at-least-once";
    Property ordering : Boolean = true;
    Property backpressure : String = "bounded";
}
Attachment ImageProcessingModule.out to BackgroundSubtractionModule.in {
    Property delivery : String = "at-least-once";
    Property ordering : Boolean = true;
    Property backpressure : String = "bounded";
}
Attachment BackgroundSubtractionModule.out to GestureRecognitionModule.in {
    Property delivery : String = "at-least-once";
    Property ordering : Boolean = true;
    Property backpressure : String = "bounded";
}
Attachment CustomizationModule.out to GestureRecognitionModule.in {
    Property delivery : String = "at-least-once";
    Property ordering : Boolean = true;
    Property backpressure : String = "bounded";
}
Attachment Real-TimeCollaborationModule.out to Single-HandInteractionModule.in {
    Property delivery : String = "at-least-once";
    Property ordering : Boolean = false;
    Property backpressure : String = "bounded";
}
Attachment ImageProcessingModule.out to Real-TimeCollaborationModule.in {
    Property delivery : String = "at-least-once";
    Property ordering : Boolean = true;
    Property backpressure : String = "bounded";
}
Attachment ThreadingModule.out to MultiprocessingModule.in {
    Property delivery : String = "at-least-once";
    Property ordering : Boolean = true;
    Property backpressure : String = "bounded";
}
Attachment MultiprocessingModule.out to LatencyOptimizationModule.in {
    Property delivery : String = "at-least-once";
    Property ordering : Boolean = true;
    Property backpressure : String = "bounded";
}
Attachment LatencyOptimizationModule.out to Real-TimeCollaborationModule.in {
    Property delivery : String = "at-least-once";
    Property ordering : Boolean = true;
    Property backpressure : String = "bounded";
}
}

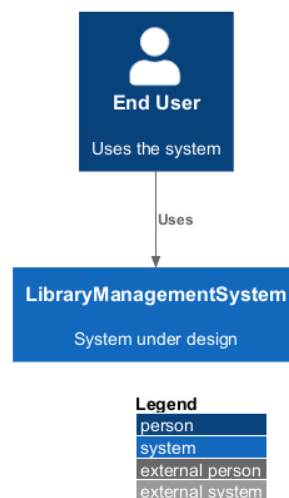
```

```

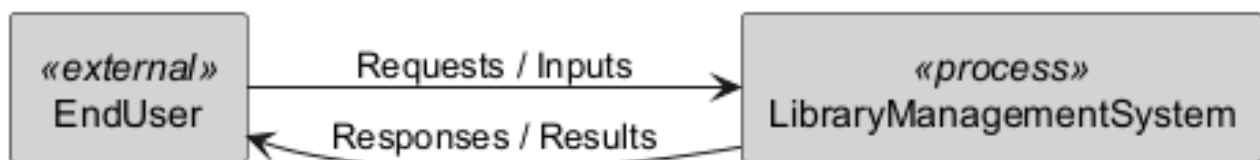
Attachment EnvironmentalAdaptationModule.out to Real-TimeCollaborationModule.in {
  Property delivery : String = "at-least-once";
  Property ordering : Boolean = false;
  Property backpressure : String = "bounded";
}
Property resilience_pattern : String = "Retry, CircuitBreaker";
Property delivery : String = "at-least-once";
Property ordering : Boolean = false;
Property authentication : String = "service-to-service";
Property authorization : String = "RBAC";
}

```

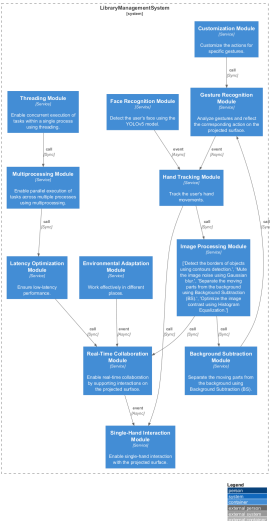
3. Context View



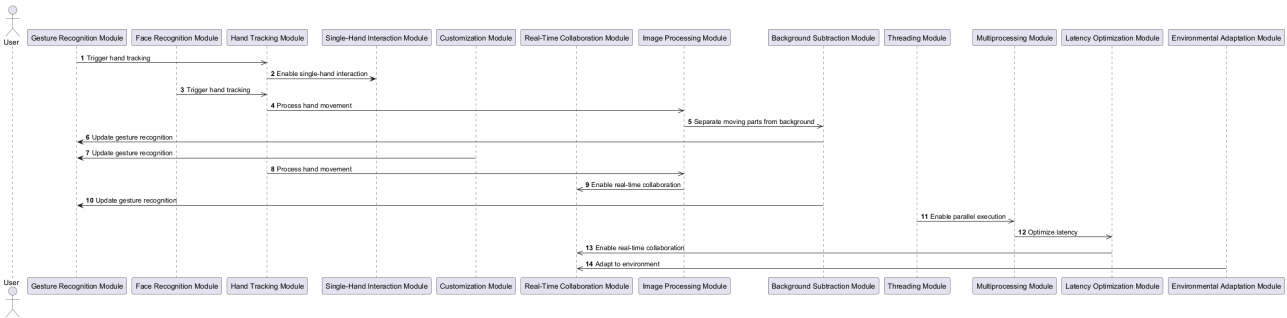
3.1 Data Context View (Level 0 DFD)



4. Logical View (C4 Container Diagram)



5. Process View (Runtime Interaction)



6. Physical View (Deployment Diagram)

